22-7-25

Ex.No.5. # Study of Activation Function and their Role

Introduction

In deep learning neural networks are built from layers of interconnected neurons. Each neuron applies a linear transformation (weighted sum of inputs plus bias), followed by a non-linear activation function. Without activation function. neural network function would be equivalent to a linear model, regardless of depth

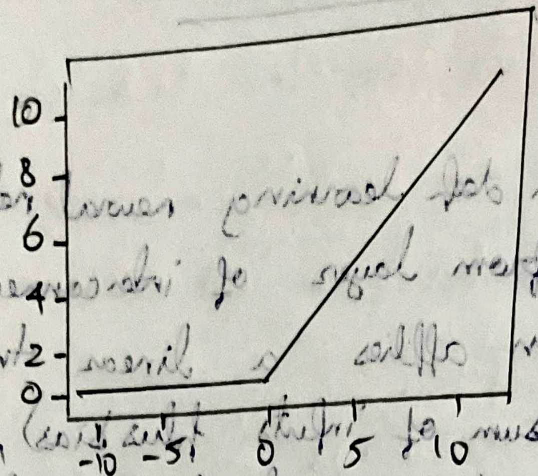
Need for Activation function:

* Introduce Non-Linearity : Real world data are highly non-linear. without activation, the neurn cannot capture such complexity

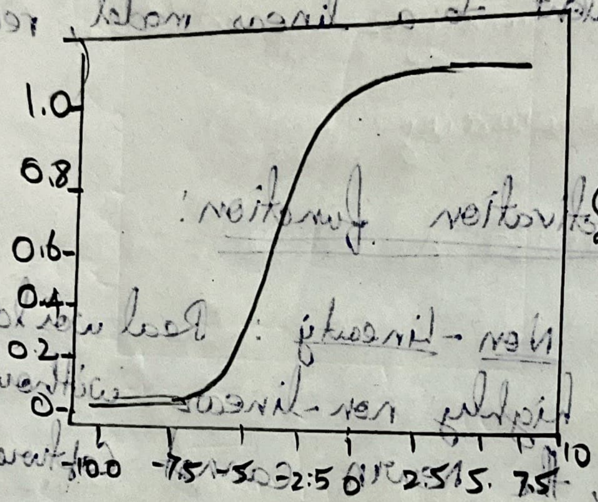
* Enable Deep Architecture: Multiple layers with activation allow networks to learn hiararchial representation

* Control Output Range: They shape neuron output (eg. bounded between 0 and 1, -1 and 1, or unbounded)

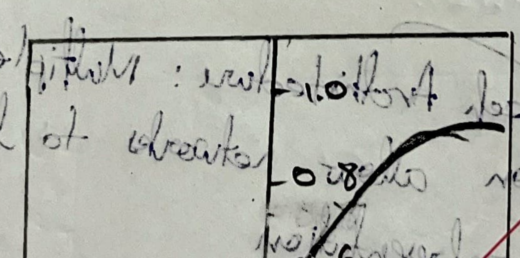
* Gradient Flow: They affect how errors backpropagate during training



Rectified Linear Unit



Sigmoid



Softmax

Common Activation Functions

(a) Rectified Linear Unit (ReLU)

$$f(x) = \max(0, x)$$

- if the input is positive it passes it through
- if the input is zero or negative, it turns it into 0

(b) Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

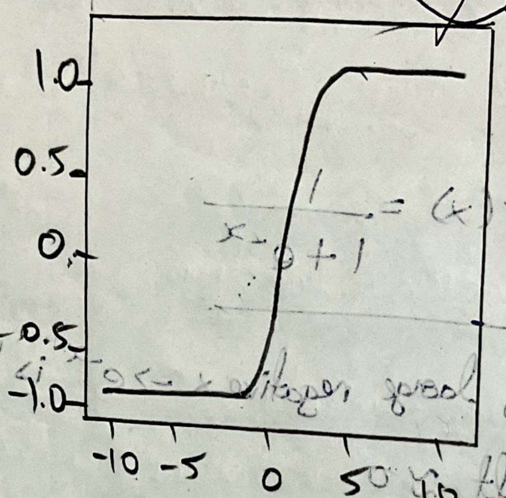
- For very large negative $x \rightarrow e^{-x}$ is hug then output ≈ 0
- For $x=0 \rightarrow$ output $= 0.5$
- 4.3 For very larg positive $x \rightarrow e^{-x}$ is tiny $\rightarrow$ output ≈ 1

Range $= [0,1]$

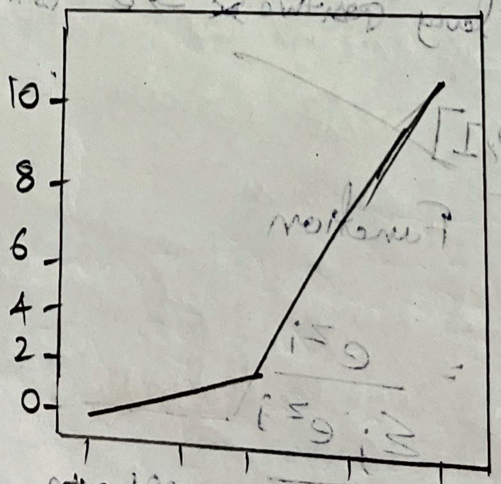
(c) Softmax Function

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

- The outputs are probabilities (sum=1)
- The largest input gets the highest probability, but others aren't discarded completely



Tan h



Leaky ReLU

Role of Activation Function

1. Add non-linearity
2. Control output range
3. Aid gradient flow
4. Make layers useful
5. Learn complex patterns.

d.) Hyperbolic tangent (tanh)

$$f(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

Range : $(-1, 1)$

↳ Centred around 0, better than sigmoid. Still suffer from vanishing gradient.

e) Leaky ReLU

$$f(x) = x \text{ if } x > 0 \text{ else } 0.01x$$

↳ Prevents dead neurons by allowing a small gradient for negative inputs

Pseudo Code

①. Initialize dataset

② Define a simple neural network with:
- Input layer
- output Layer
- hidden layer

3. choose an activation function for the hidden layer (sigmoid, tanh, ReLU, Leacky Relu, Softmax)

4. For each activation function:

   a. Assign the activation function to the network

   b. Initialize weights and biases

   c. Define loss function

   D. Define optimizer

   e. for fixed number of epochs:

      i, Forward pass

      ii, Compute loss

      iii, Backward pass

      iv, update weights

   f. evaluate the trained model on test data

Observation:-

→ Activation functions introduce non-linearity

→ They control the flow of information through the network by transforming weighted inputs

→ The choice of Activation function affects training speeds gradient flow, and final accuracy

Result:- Activation functions are essential in deep Learning as they introduce non-linearity enabling neural network to learn complex patterns.